TRAINING · SOURCE-ARCHIVE DEEP DIVE

NORD Time Sharing System

An introduction to TSS by Bo Lewendal — kernel, user space, and every device down to the register bits — ending at the deepest layer: the NORD-10 swapping-drum driver.

TSS1–TSS5 · MAC ASSEMBLER NORD-1 / NORD-10 DRUM DRIVER: NJL · 17 APR 1973

INTRODUCTION

What TSS Is

01 A complete, **standalone timesharing operating system** for the NORD-1 and NORD-10. It owns the machine — it is not a program running under SINTRAN.

02 Written by **Bo Lewendal** in MAC assembler, in five source parts, with the algorithms documented in structured pseudo-code comments beside the code.

03 Every terminal is a **process** with its own 8-page virtual memory, swapped between core and mass storage — CDC disk, or a drum.

04 Users get a command processor, a file system, mail, accounting — and programs get a monitor-call interface.

5
source parts, TSS1–TSS5

≈14,000
lines of MAC assembler

81
monitor calls in MCTBL

24
teletypes maximum

INTRODUCTION

The Source Archive

FILE	CONTENTS	LINES
TSS1:SYMB	Kernel — interrupts, all device drivers, scheduler, swapper, page tables	4,709
TSS2:SYMB	Context block, monitor-call implementations, command processor	3,512
TSS3:SYMB	Overlay machinery (GOVER / OVERL / OVERX) and system utilities	1,755
TSS4:SYMB	Utility overlays — DUMP, RECV and friends	2,040
TSS5:SYMB	User-management overlays — create / delete user, accounts	1,995
ASSYSA / ASSYSB	MAC build scripts — version A, and version B with DEBUG mark	25 ea.
MINIT / TDUMP	Standalone mass-storage initializer and TSS dumper	810 / 383

The archive also holds the build *outputs*: LIST1–LIST5 listing streams, plus ASYMB / BSYMB — complete symbol-table dumps that serve as a golden oracle for any re-assembly.

INTRODUCTION

The Big Picture

USERS	**Up to 24 teletypes**	one logged-on user → one process
USER SPACE · S02	**8 × 2K pages @ 40000_8**	command processor · utility overlays · user programs — swapped
KERNEL · S01	**Resident TSS**	scheduler · swapper · monitor calls · buffers · file system · drivers
DEVICES · S03	**TTY · tape · cards · printer · CDC disk**	registers & bits, via IOT / IOX
THE DRUM · S04	**XDRUM @ IOX 540_8**	the swap device — full register & bit walkthrough
BUILD & RUN · S05	**FMAC + ASSYSA**	marks → assemble → save to disk → boot standalone

01

The Kernel

Resident TSS: interrupt levels, memory management, scheduler, swapper, monitor calls, buffers, and the file system.

SECTION 01 · KERNEL

Interrupt-Level Architecture (NORD-10 build)

14	Traps — illegal instruction, protect violation, monitor-call decode
13	Real-time clock
12	Input interrupts — TTY in, reader, cards (IDENT PL12)
11	DMA devices — Versatec printer option
10	Output interrupts — TTY out, punch, line printer
9	Subsidiary clock — quanta, timeouts, response timers
6	Teletype scanner — polls dead terminals every 80 ms
5	**Scheduler + swapper** (LEV5 → SWAPR)
3	Monitor-call processor — dispatch through MCTBL
2	Utility level — command processor entry, user services
0	Idle loop — traps WAIT, hands control onward
	NORD-1 build: levels 7 / 11 and IOT instructions instead

False interrupts and I/O errors are counted per level in NER10 / NER11 / NER12 / SER10 / SER12 / SER23 / NIOX cells — built-in diagnostics for flaky hardware.

SECTION 01 · KERNEL

Memory Management

VCTBL

Virtual core table — the 8 pages the current process *wants*: mode, swap index, disk address.

PCTBL

Physical core table — what actually sits in each of the 8 core page frames right now.

PDTBL

“Physical drum table” — the swap-space allocator: usage count, mode, file address per swap page.

Page modes drive swap policy: `MD=1` scratch (never written back), `MD=2` read-only — **shared between processes** via a reference count in PDTBL, `MD=3` read/write (written out on every switch).

User space: `NPGS=8` pages of 2K words at `MSTRT=40000`$_8$. Swap pool: 36–72 pages, scaled by the TEL*n* mark.

SMPR rebuilds protection after every switch: memory-protect registers on the NORD-1 (2K / 4K blocks) — real **paging registers at 177400**$_8$ on the NORD-10 (IPGTB sets them up at boot).

A process's registers live in its **context block** (RBLOK), stored as swap page 0 — 8 sectors at `DKA1 + index×8`.

Scheduling

```
% PROCESS TABLE (PRT) — 8 WORDS / PROCESS

PRT1, 0 %PDTBL INDEX OF CONTEXT BLOCK
  0 %QUANTUM TIMER
  0;0 %COMPUTE TIMER
  0 %STATUS: BLOCK·RUBOUT·I/O BITS
  0 %TTY USAGE (<0 DEAD, 0 INIT, >0 ALIVE)
  0 %FREE WORD
  0 %SAVED PID

QUANT, -17 %QUANTUM (NEG. TICK COUNT)
LQUA,  24 %SMALL QUANTA PER LARGE QUANTUM
```

The subsidiary clock (level 9) increments the running process's **quantum timer** each tick and wakes level 5.

LEV5 scans PRT **round-robin**. The incumbent keeps core until it blocks, dies, or burns LQUA small quanta; then the next runnable process wins.

A context switch is a call to **SWAPR** — scheduling and swapping are one mechanism, because switching processes *means* swapping memory.

Swapping — SWAPR

1. Write the outgoing process's **context block**: `TRSFR(WRITE, @RBLOK, DKA1+(PRT[0]<<3))`
2. Read the incoming process's context block, restore its registers
3. For each of 8 pages: **write out** conflicting read/write pages `(MD=3)`, keep matching ones, delete abandoned ones
4. **Read in** required pages — a shared read-only page already in the pool is reused by bumping `PDTBL[j]$COUNT`, never copied
5. Every transfer goes through one routine: **TRSFR** — 2K words between core and swap space. *That routine is where the drum plugs in (S04).*

SECTION 01 · KERNEL

Monitor Calls

A user program executes a monitor-call opcode → **trap on level 14** → the call number indexes MCTBL → the handler runs on **level 3** with the user's registers.

Three return styles: normal, skip return (success/failure), and double-skip — the caller places its error handling right after the call, like the device drivers do.

Level 14 also **emulates missing hardware**: LBYTE, SBYTE, register multiply / divide, EXRG — so one binary runs on differently-equipped machines.

```
% MCTBL — 81 ENTRIES, BY THEME

CHAR I/O   INBT OUTBT ECHOM BRKM
FILES      OPEN CFILE ISIZE OSIZE
DISK       RDISK WDISK RPAG WPAG
STRINGS    SETUP GCI WCI TRIM CNS
USERS      GUNUM GUNAM WHUSR EXEC
MAIL       IMAIL GMAIL BROAD MSG
TIME       RKLOK RDATE TTIM RCTIM
CONTROL    LEAVE RESUM ESCTR DBRK
```

SECTION 01 · KERNEL

Buffers & the Driver Pattern

Every character device owns a **ring buffer** (two 8-bit chars per 16-bit word), created by IBUF and reached only through WBUF / RBUF.

Drivers split in two halves that meet in the buffer: an **interrupt half** on the device level (TTYDI, READR, CARD1, PNTDR…) and a **process half** called via monitor calls (RTTY, WTTY, RREA, WPRNT…).

Skip-return convention: a routine returns to the instruction after the call on failure, skips one on success — callers write their error handling in line, no flags.

A **busy flag** per buffer marks a transfer in flight; a full output buffer *blocks* the process (PRT status bit) until the drain interrupt wakes it.

```
% OUTPUT PATH, e.g. WTTY
PROCESS: WTTY → BUSY?
  NO  → IOX WDR (START DEVICE)
  YES → WBUF (QUEUE CHAR)
  FULL → BLOCK PROCESS

% INTERRUPT (LEVEL 10)
TTYDO: RBUF → IOX WDR
  EMPTY → BUSZR, UNBLOCK

% INPUT MIRRORS IT ON LEVEL 12
TTYDI: IOX RDR → WBUF → BRKCH
```

SECTION 01 · KERNEL

The File System

Storage past FSYS is allocated in **tracks** through a bit table (BITBL, 400_8 words) in the master information block — GTRK grabs a track, RTRK returns it.

The **user table** (USTBL) maps users to their file directories; OBT tracks up to **NOPEN=16 open files** (64 words of state each).

Every file-system transfer moves **2K words** through TRXX → the CDC disk driver — the same size as a swap page.

```
% DEFAULT FILE TYPES
'BIN'  BINARY   'PROG' DUMPS
'BRF'  RELOC.   'SYMB' SOURCE
'DATA' DATA     'CORE' SAVES
'RB'   RB FILES
```

In-core caches with lock words: BITBL / USTBL / BTEMP each carry LOCK · IN-CORE FLAG · DEVICE ID headers — one 256-word disk sector cached at a time.

02

User Space

Command processor, utility overlays, and user programs — the swapped world above 40000_8.

SECTION 02 · USER SPACE

Logon & the Command Processor

A keypress on a **dead terminal** wakes the level-6 scanner → the scheduler initializes the process → SWAPR's cold path (S10) acquires a context block, sets **logon mode**, and loads the command processor from the **coreload** area (CORL1 / CORL2).

Commands parse through the string machinery (CMDLN, GCMD, PCF) and either run in-line or pull a **utility overlay**.

RUBOUT / ESC is sacred: it fires the break logic, RSLK releases every lock, and the utility restarts — the guaranteed way out of anything.

```
% PER-PROCESS STATE (TSS2)
CMDLN  COMMAND LINE BUFFER
RUBAD  ESCAPE TRAP ADDRESS
RUBPR  SAVED P AT RUBOUT
RMODE  0 NORMAL / -1 LOGON
OVLAY  CURRENT USER OVERLAY
PRNUM  PROJECT NUMBER
```

TSS measures itself: RSTIM / RSNUM accumulate **response time per break character** — 1970s telemetry.

SECTION 02 · USER SPACE

Utility Overlays & User Programs

Utilities live as **1000_8-word overlays** in OVDK on disk. Calling one is a macro: GOVER traps to the loader, which swaps the overlay into the ROVER window and jumps in.

The build trick: OVERX **writes each overlay to disk while MAC is still assembling** — assembly and installation are the same act.

TSS4/TSS5 overlays: DUMP · RECV · CRUSE · DLUSR · account and mailbox maintenance.

```
% USER PROGRAM LIFE CYCLE
RECV  'PROG' FILE → PAGES+REGS
RUN   SWAPPED LIKE ANY PROCESS
MON   CALLS INTO MCTBL
DUMP  PAGES+REGS → 'PROG' FILE
```

DUMP stores the full machine state — registers, RUBAD, a magic word 131313_8 — so RECV resumes a program exactly where it stopped.

03

The Devices

IOT and IOX at the bit level — then terminal, tape, cards, printer and CDC disk, register by register.

SECTION 03 · DEVICES

Talking to Hardware: IOT and IOX

"NN10 — NORD-1 · IOT = 160000_8 + bits

15–13 OPCODE 7_8	11 PIN 2000_8	9 SKA 1000_8	8 ACT 400_8	7–0 DEVICE NO.

```
ACT ACTIVATE / TRANSFER DATA
SKA SKIP NEXT IF DEVICE READY
PIN PERMIT (ENABLE) INTERRUPT
SNI SKIP IF NO INTERRUPT (SCAN)

IOT ACT REA  % READ TAPE CHAR → A
IOT PIN ACT DFP % PUNCH A + INT
```

"N10 — NORD-10 · IOX / IOXT

15–11 OPCODE	10–1 · DEVICE REG. ADDRESS	0 · R/W EVEN/ODD

```
EVEN ADDR → DEVICE READS INTO A
ODD  ADDR → A WRITTEN TO DEVICE

IOX REA RDR   % 400 → READ CHAR
SAA 5; IOX DCR WCR % 423 ← CONTROL
IDENT PL12    % WHO INTERRUPTED?

IOXT: REG. ADDRESS TAKEN FROM T
→ FULL 0–177777₈ ADDRESS RANGE
```

Every TSS driver is written twice inside "NN10 / "N10 conditional blocks — one source, two machines. TSS uses plain IOX throughout; device numbers stay below 2000_8.

SECTION 03 · DEVICES

The Standard Device: 4 Registers (NORD-10, base + 0…3)

+0 RDR	read data → A
+1 WDR	write data ← A
+2 RSR	read status → A
+3 WCR	write control ← A

```
% CONTROL VALUES IN TSS
5  = INT + ACTIVATE (TTY,TAPE,LP)
7  = INT + ERR-INT + ACT (CARDS)
3  = INTS ONLY, NO ACTIVATE
20 = DEVICE CLEAR (ERROR RECOVERY)
0  = DISARM (QUIESCE DEVICE)
```

STATUS REGISTER (RSR) — BITS AS TESTED BY TSS

bit 0	interrupt enabled (readback of WCR bit 0)
bit 3	**device ready for transfer** — polled as `BSKP ONE 30 DA`
bit 4	**inclusive OR of errors** — `BSKP ZRO 40 DA` guards every driver
bit 5+	device-specific (framing / parity / card-done …)

CONTROL REGISTER (WCR)

4	3	2	1	0
DEV.CLEAR	TEST	ACTIVATE	INT ON ERR	INT ON RDY

Bit numbers in ND notation: `BSKP … 30 DA` = bit 3, `40` = bit 4, `170` = bit 15 (octal tens = bit index).

Terminals IOX 300_8+10_8n · in = base, out = base+4

300 / 304	TTY1 read data / write data
302 / 306	TTY1 read status (in) / (out)
303 / 307	TTY1 write control (in) / (out)
…1370	TTY9–16 at 1300–1370

STATUS BITS USED (RSR)

bit 3	char received / transmitter ready
bit 4	error OR → counted in SER10/12/23
WCR←5	arm interrupt + activate after each char

```
% INPUT FLOW (TTYDI, LEVEL 12)
IOX ttyi RSR  → BIT 4 SET? → TERR
IOX ttyi RDR  → CHAR → CKPAR PARITY
WBUF: RING BUFFER, 2 CHARS/WORD
BRKCH: BREAK STRATEGY 0/1/2/USER
ECHCH: ECHO NOW / DEFERRED / NONE
IOX ttyi WCR ← 5 (REARM)
```

Break decides which character wakes the process; **echo** can run at interrupt level (full duplex) or be deferred into an echo buffer so program output stays coherent. Buffer descriptors carry both tables per terminal (words 11–30 of each BUFTB entry).

Paper Tape & Card Reader

TAPE READER 400_8 · PUNCH 410_8

400 RDR	read tape character → A
402 RSR	bit 3 char ready · bit 4 error
403 WCR	← 5 read next char + interrupt
411 WDR	punch character ← A, then WCR←5

A stuck reader is released by the **5-second timeout** (TIMOUT=-372_8 ticks) on the clock level; buffers: 9PPT=600_8 in, 9FP=600_8 out.

CARD READER 420_8

420 RDR	read column code → A
422 RSR	bit 3 column ready · bit 4 exception · bit 5 card done · bit 9 fault
423 WCR	← 7 = ready-int + error-int + activate · ← 20_8 clear on fault

A card buffers into 121 words (CTBB), converts through the 1971 CTB1/CTB2 card-code→ASCII tables, gets CR LF appended — a full card errors as 25_8 (NAK) if the process lags.

Line Printer IOX 430_8 · driver PNTDR / WPRNT

431 WDR	print character ← A
432 RSR	bit 3 ready for next char · bit 4 error OR (offline, paper out)
433 WCR	← 5 print + interrupt · ← 20_8 device clear, then retry char

```
% ERROR RECOVERY (WPRNT, N10)
IOX DLP RSR; BSKP ZRO 40 DA
  JMP TERR  % BIT 4 → ERROR
TERR, SAA 20; IOX DLP WCR
  SHA ROT 20; JMP L21 % CLEAR+RETRY
```

WPRNT substitutes unprintable characters and expands LF handling; output is buffered through 8LP=200_8 words and owned by one terminal via the LPNT lock.

The **Versatec option** replaces character output with DMA: VERDR fills a 132-char line buffer, loads its address (IOX DLP+1), count 1 line (DLP+3), go (DLP+5 ← 5) — completion interrupts on **level 11**.

CDC Disk IOX 500_8–507_8 · driver DKTR

500 RCA	read core address
501 LCA	load core address (DMA target)
502 RSECT	read sector counter
503 LBA	load block address — cyl · head · sector packed by DKADR
504 RST	status: **bit 2** ready · **bit 4** error OR · **bit 14** op complete
505 LMR	modus: **bits 14–13** op (rd/wr/par/cmp) · **bit 2** activate · **bit 4** device clear
506 SEEK	position unit
507 LWC	load word count (sectors × 256)

DKTR: X = core addr, T = linear disk addr (256-word sectors), A = op + unit, D = extra sectors (≤3K words). Two retries, then error with the status word in A.

DKADR maps the linear address ÷12 onto cylinders (313 / 626 for CDC 9425 / 9427); unit 1 sits at +52600_8.

The swapper's page ops and the file system's 2K transfers (TRXX) both end here when no drum is fitted.

DISK LAYOUT (FIXED ADDRESSES FROM DKBIT=50_8)

04

The Swapping Drum

XDRUM — the NORD-10 drum driver, every register and bit, and the TRSFR front end that routes swap traffic between drum and disk.

“1. APPROXIMATION TO A NORD-10 VERSION” — NJL, 17/4/73

SECTION 04 · THE DRUM

Drum Basics & Geometry

A drum is a rotating magnetic cylinder with **one fixed head per track** — no seek arm, no seek time. Latency is rotational only, which made it *the* swap device of its era.

The controller is a **DMA block device**: load core address, block address and word count, start it, take an interrupt when done.

TSS ran on the NORD-1 too — but the drum driver itself is **NORD-10 code only**: pure IOX, guarded by `"DRUM N10`, dated 17/4/73.

Sector (block)	**64 words**
Sectors per track	**32**
Track = 32 × 64	**2,048 w = 1 TSS page**
Default size (DRMSZ)	**2000_8 × 256 w**
= total capacity	**256K words · 128 tracks**

SECTION 04 · THE DRUM

TRSFR — Drum or Disk?

```
"DRUM N10  % ONLY WITH BOTH MARKS
TRSFR, STF TRSAV; STX TRSX; ...
TRSF1, LDA TRSAV; SUB (DKA1; SHA 2
  COPY DD SA  % D = (T-DKA1)*4
  SUB (DRMSZ*4; JAN TRSF2% FITS?
  LDA TRSAV; SUB (DRMSZ; ...
  JPL TRXX% NO → DISK @ T-DRMSZ
TRSF2, LDT TRSAV+1; SHT ZIN SHR 1
  SAX 40% 32 BLOCKS = ONE 2K PAGE
  JPL XDRUM
  JMP TRSF1 % ERROR → RESTART ALL
  JMP *-2   % BUSY → CALL AGAIN
TRSF3, ... JMP I TRSVL % FINISHED
```

D = (T − DKA1) × 4

disk sectors are 256 words, drum blocks 64 — factor 4

First DRMSZ pages of swap → drum.

Beyond → CDC disk at T−DRMSZ via TRXX / DKTR.

Op translation: swap ops 1/3 (read/write) shift right → drum functions 0/1.

Without the DRUM mark: TRSFR=TRXX — disk only.

SECTION 04 · THE DRUM

XDRUM — the Programming Interface

```
% CALLING SEQUENCE
  JPL I (XDRUM
  JMP ERROR  % X=STATUS REG
  JMP BUSY   % RE-CALL, REGS UNCHANGED
  JMP FINIS  % X=STATUS, A=CORE ADDR
```

Busy-exit protocol: call again at once, or after the drum interrupt, with X T A D untouched. The B register is never used.

X	Number of blocks (64-word sectors) to transfer
T	Function — 0 read · 1 write · 2 read test · 3 compare · 20_8 read status only. **Bits 8–9 = core-address bits 16–17** (>64K core!)
A	Core address (low 16 bits)
D	Block (drum) address — linear sector number
→X	On error *and* on finish: the drum status register comes back in X

XDRUM — Registers & Bits IOX 540_8+n

540 RCX	read core address (DMA pointer)
541 LCX	load core address
543 LBX	load block address (track · sector)
544 RSX	read status register
545 LCR	load control register (go!)
547 LWX	load word count = blocks × 64

STATUS REGISTER (RSX 544)

bits 4–0	echo of current function code (masked 37_8)
bit 4 DVA=20_8	**device active** → busy exit
bit 5 ERR=40_8	**inclusive OR of errors** → error exit

CONTROL REGISTER (LCR 545) — AS ASSEMBLED BY XDRUM

15	14–13 FUNCTION 0–3	12–7 —	6–5 CORE A17–A16	4–3 —	2–0 = 7 INT EN + ACTIVATE

```
SAA 3; AND DTREG; SHA ZIN 13 % FCN→13-14
LDA (1400; AND DTREG; SHA ZIN SHR 3 % A16/17→5-6
AAA 7; IOX DRM LCR % +INT+ACT → GO
```

BLOCK ADDRESS (LBX 543) — FROM LINEAR SECTOR NO.

15–11 SECTOR 0–31	10–0 TRACK

The driver splits D into track (D÷32) and sector (D mod 32) with the ROT-13 / ROT-5 shuffle at DRDLC.

Error path: X ← status, then `SAA 4; IOX 545` — control ← 4 = device clear by the standard convention *[assumed — no controller manual]*.

Address & Count Registers 540 · 541 · 543 · 547

541 LCX write

540 RCX reads it back

15–0 · CORE ADDRESS (word address, DMA target/source)

Only 16 bits wide — core-address bits 16–17 go in the **control register** (next slides). RCX exists for diagnostics; XDRUM never reads it.

547 LWX write

word count

WORDS TO TRANSFER = CHUNK × 64 (64 … 2048)

Driver: SHA ZIN 6 (shift left 6 = ×64). Max one full track: 32 sectors = 2048 words. Whole 64-word sectors only — there is no partial-sector transfer.

543 LBX write

hardware block address

15–11 · SECTOR 0–31	10–0 · TRACK (0–127 on the default drum)

```
SAD ZIN 13; SHA ROT 13; RADD SA DD  % FROM LINEAR D:
SHA ROT 5; SAD 5  % SECTOR=D MOD 32 → 15-11, TRACK=D/32 → 10-0
```

Status Register 544 RSX · read

15–6	5	4	3–0 (& 4)
controller-specific — undocumented, returned to caller in X	5 · ERR =40_8	4 · DVA =20_8	FUNCTION ECHO · mask 37_8

bit 4 · DVA	**Device active** — operation in progress. Driver: `BSKP ZRO DVA DA → JMP DBUSY` — restore registers, take the **busy exit**, caller re-calls after the interrupt.
bit 5 · ERR	**Inclusive OR of all errors**. Driver: `BSKP ZRO ERR DA → JMP DERRO` — status → X, device clear, **error exit**. Individual error causes (parity, timing, address) are controller detail — no manual survives.
bits 4–0 echo	Readback of the last **function code**. Driver: `SAA 37; RAND ST DA`, then `AAA -20; JAZ DRDS1` (was it READ STATUS = 20_8? → finished) and `AAA 14; JAP DRILL` (> 3 → illegal function, error exit with X = −1).
whole word	Returned in **X** on both the error and the finished exit — the caller sees exactly what the controller reported.

Control Register 545 LCR · write = GO

15	14–13	12–7	6–5	4	3	2	1	0
—	FUNCTION	—	CORE A17–A16	DEV.CLR*	—	ACTIVATE	ERR INT*	RDY INT

FUNCTION CODES (BITS 14–13)

0	**READ** — drum → core (page in)
1	**WRITE** — core → drum (page out)
2	**READ TEST** — read & check, no core transfer: surface / parity verify
3	**COMPARE** — read & compare against core: write verify
20_8	READ STATUS ONLY — software code, never written here: XDRUM answers it from RSX and exits finished

```
% HOW XDRUM BUILDS THE WORD
SAA 3; AND DTREG  % FCN = T & 3
SHA ZIN 13        % → BITS 13-14
LDA (1400; AND DTREG % T BITS 8-9
SHA ZIN SHR 3     % → BITS 5-6 (A16/A17)
RORA ST DA; AAA 7 % + INT EN + ACTIVATE
IOX DRM LCR       % GO
```

Value 7 (bits 0–2) = “enable interrupt and activate” per the source. **Value 4** on the error path = device clear / abort. Bits marked ***** are convention, unconfirmed.

A17–A16 (bits 6–5): extend LCX to 18 bits — DMA into 256K words of core.

XDRUM — Transfer Flow & Track Splitting

1	IOX DRM RSX — DVA set → **BUSY exit** · ERR set → **ERROR exit**
2	X = 0, or function = 20_8? → read status, **FINISHED exit**
3	Chunk = **min(X, 32 – (D mod 32))** — never cross a track boundary
4	Load word count (chunk×64), core address, packed block address
5	Advance the saved registers: X−=chunk, D+=chunk, A+=chunk×64
6	Write control word (function + A16/17 + 7) → hardware runs → **BUSY exit**; caller re-calls at step 1

ONE 2K-PAGE SWAP = ONE TRACK

32 sectors × 64 w
single operation

An unaligned transfer becomes several operations, one busy-exit round trip each.

Interrupt-ready by design: the busy exit lets the caller sleep on the drum interrupt instead of polling — TSS's TRSFR simply polls (JMP *-2), a “first approximation”.

05

Build, Install, Run

Library marks → FMAC assembly → save to disk → boot a standalone TSS.

SECTION 05 · BUILD & RUN

Configuration: Library Marks

NCR / CDC	disk system flavor (CDC here)
DRUM	**compile the swapping-drum path** (default off)
N10	**NORD-10 version**: IOX I/O, paging, instruction emulation
DEBUG	version B instead of A (separate overlay/coreload areas)

K12/K14/K16	resident kernel size: 12K / 14K / 16K
TEL4…TEL24	number of terminals (4–24)
DRMSZ	drum size in 256-word pages (default 2000_8)
9TTI…9CR	buffer sizes; TT1…TT12, DLP, DCR… device numbers

```
AS ARCHIVED: CDC MACF DIAB K14 TEL4
```

→ NORD-1 IOT I/O, disk-only swapping. For the drum system this deck describes: **`CDC MACF K14 TEL4 DRUM N10`**

SECTION 05 · BUILD & RUN

The Build: ASSYSA, Decoded

```
FMAC              % START THE ASSEMBLER
100 / SYSA        % PROMPTS: SIZE, NAME
CDC MACF DIAB K14 TEL4 % THE MARK LINE
IOT=160000;ACT=400;... % CVS RESTORATION:
INTDS=150440;...      % SYMBOLS MAC HAD BUILT IN
)9ASSM TSS1,LIST1,0   % ASSEMBLE PART 1
)9ASSM TSS2,LIST2,0
)9ASSM TSS3,LIST3,0
)9ASSM TSS4,LIST4,0
)9ASSM TSS5,LIST5,ASYMB:SYMB % + OBJECT STREAM
)LIST             % DUMP SYMBOL TABLE
*: / ? / )9TSS       % NOT IN ND-60.096 [UNKNOWN]
```

TSS assembles **into memory on the target machine** — the overlay machinery even writes overlays to disk *during* assembly (`)WRTM / )SOVER`).

The listing streams (LIST1–5) and symbol dumps (ASYMB / BSYMB) in the archive are this script's actual output — a **golden oracle**: any re-assembly must reproduce those symbol values.

ASSYSB is identical but sets DEBUG, names it SYSB, skips listings → BSYMB.

SECTION 05 · BUILD & RUN

Install & Boot

1	**MINIT** initializes mass storage: MIB bit table, user table, empty file system
2	Run ASSYSA under FMAC — TSS is now **assembled in memory**, overlays already on disk
3	Execute location 7: `LDT *+3; JMP I *+1; SYSSV; CORLD` — **SYSSV writes 32K words** ($100_8 \times 400_8$-word blocks) to the CORLD coreload area, then falls into INIT
4	**INIT**: interrupt vectors, paging (IPGTB), buffers, links, FINIT file system, enable levels — idle loop waits for a keypress
!	TSS **replaces** the machine's OS — on an emulator, boot the saved coreload instead of SINTRAN; the master terminal logs on as SINIT when OPR = 131313_8

CLOSING

Open Questions & Next Steps

UNKNOWN

- No **MAC / FMAC binary** in the archive — the one missing tool
- `*:  ?  )` 9TSS at the end of ASSYSA — not in ND-60.096.01
- Drum status bits beyond DVA / ERR — controller manual needed
- XDRUM is a self-described **“1. approximation”** — review before trusting

NEXT STEPS

1 Locate a MAC / FMAC tape image → authentic build under nd100x / RetroCore

2 Failing that: MAC-dialect cross-assembler, verified against **ASYMB / BSYMB**

3 Build `CDC MACF K14 TEL4 DRUM N10` → drum-swapping NORD-10 TSS

4 Exercise XDRUM against the restored drum hardware — carefully